Quantum Variational Algorithms on Adabatic Quantum Computing Devices

Philipp Hanussek

July 13, 2026

1 Overview

Since the development of the quantum computers, many claims regarding quantum advantage have been made. Often, these advantage claims did not last long. In this work, we focus exclusively on the D-Wave Quantum Annealer, which leverages quantum technology to solve quadratically binary unconstrained optimization (QUBO) problems. D-Wave is among the first companies to commercially sell quantum computers. In 2025, researcher from D-Wave claimed to have solved a scientifically relevant problem fast using quantum technology, than it would have been possible on classical devices. However, it is currently an open question whether the D-Wave Quantum Annealer is able to solve other problems well, and possible faster than convential computers. In this work, we assess the D-Wave's ability to sample from a probability distribution, to approximate the ground state of different statistical mechanical models using variational algorithms.

1.1 Previous work & motivation

We give an overview of the current research on quantum computing devices and present our motivation for using variational algorithms to simulate quantum systems.

1.1.1 Quantum Annealing Performance

The D-Wave's Quantum Annealers performance scaling for finding the ground state (corresponding to the minimum of the quadratic binary cost function) has been extensively studied, in the area of cryptography and physical simulation. In general, classical methods exhibit faster solving times and better scaling. We briefly report on previous work in the field of simulating physical systems using quantum annealing hardware. In [**?**], we used a method presented in [**?**] to encode the solution to the wave function of different Hamiltonians into a QUBO problem. These instances were then solved on different physical und physics-inspired solvers. We evaluate the scaling of the samplers using the time to solution (TTS) metric, which corresponds to the computational time needed to solve the optimisation problem with a success probability of at least P_{target}. The TTS is defined as

$$\text{TTS} = \frac{\ln(1 - P_{\text{target}})}{\ln(1 - P_{\text{success}})} T_r, \tag{1}$$

where P_{success} is the probability of obtaining the ground state in a single run, and T_r denotes the runtime of a single run.

It is important to note that current D-Wave devices exhibit limiting connectivity. As a consequence, most problem instances need to be adapted in an additional *embedding* step, increasing the number of physical qubits required. We compare two problem formulations, one advantageous for the quantum annealer, as no extra embedding step is required, and a second one after embedding. Classical devices dominate in both cases (see **??**).

1.1.2 Motivation

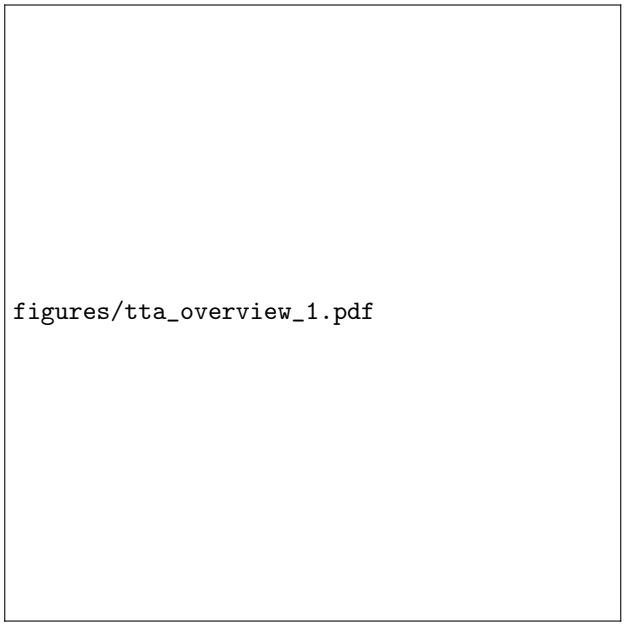
As previously shown, results return by the quantum annealer are inherently noisy. This makes it difficult to sample the *exact* ground state of the submitted Hamiltonian. In this work, we therefore assess the device's ability to act as a Gibbs sampler and sample from the Boltzmann distribution. It has been shown earlier that the D-Wave device can in fact be used as a Gibbs sampler[**?**][**?**]. Here, we systematically evaluate the quality and time of these samples on computationally hard problems, which allows us to rigorously compare the quantum annealer to classical physics inspired algorithms.

2 Methods

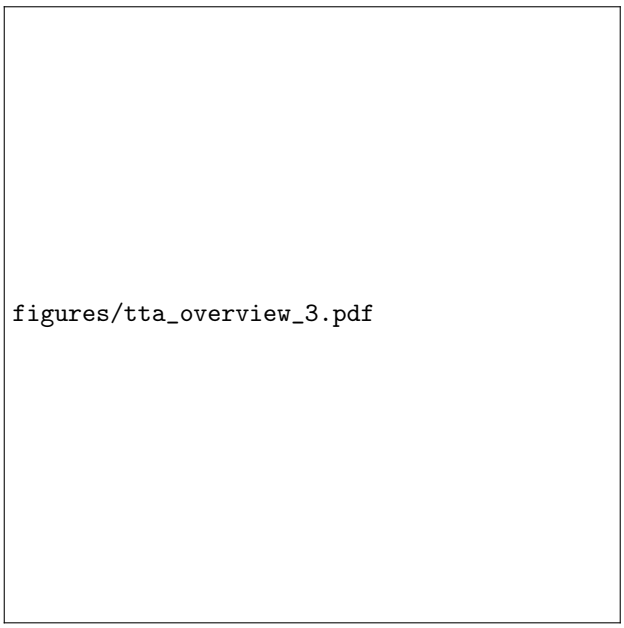
In this section we briefly outline the simulated physical systems as well as the main computational and algorithmic methods used in this work.

2.1 Quantum Ising Models

The Ising model consists of discrete spins arranged on a lattice, interacting via nearest-neighbour couplings. In its quantum extension, the Transverse Field Ising Model (TFIM), a transverse magnetic field is added, introducing quantum fluctuations through non-commuting terms in the Hamiltonian.



(a) D-Wave native formulation. This corresponds to a single-qubit Y rotation, governed by $H_1 = \frac{\pi}{2}\sigma_y$.



(b) Non-native formulation. This corresponds a single-qubit Rabi oscillations driven along y, described by $H_3 = \pi\left(\frac{3}{4}\not{\mathbb{1}} - \frac{1}{4}\sigma_y\right)$.

Figure 1: Previous work on sampling the exact ground state to simulate quantum systems. TTS99 vs. N: exponential scaling $\propto \exp(N/\beta)$. GPU solvers dominate for $N \gtrsim 100$; missing points indicate zero success probability.

2.1.1 Transverse Field Ising Model

The Hamiltonian for the one-dimensional TFIM is given by

$$H = -J\sum_i \sigma_i^z \sigma_{i+1}^z - h\sum_i \sigma_i^x \tag{2}$$

where J is the coupling strength and h is the transverse magnetic field strength. The 1D TFIM serves as a benchmark for our models, as it is exactly solvable via the Jordan–Wigner transformation. Despite its simplicity, it displays non-trivial quantum behavior, including a quantum phase transition and quantum fluctuations.

2.1.2 Long-range Transverse Field Ising Model

The Hamiltonian for the one-dimensional long-range transverse-field Ising model (LRTFIM) is given by

$$H = -\sum_{i<j} \frac{J}{|i-j|^\alpha}\sigma_i^z \sigma_j^z - h\sum_i \sigma_i^x \tag{3}$$

where J sets the interaction strength, α controls the power-law decay of the interactions, and h is the transverse magnetic field strength. The model serves as an extension of the 1D-TFIM and allows us to assess the ability of our models to capture long-range interactions.

2.2 Heisenberg Models

The Heisenberg model describes spins on lattice sites interacting via exchange coupling:

$$H = \sum_{\langle i,j\rangle} J_{ij}\,\boldsymbol{\sigma}_i \cdot \boldsymbol{\sigma}_j \tag{4}$$

where $\boldsymbol{\sigma}_i = (\sigma_i^x, \sigma_i^y, \sigma_i^z)$ are the Pauli spin operators, J_{ij} is the exchange coupling between sites i and j, and $\langle i,j\rangle$ denotes nearest-neighbour pairs. In contrast to the TFIM, the Heisenberg model couples all three spin components.

2.2.1 XXZ Heisenberg Chain

We study the one-dimensional XXZ Heisenberg chain with periodic boundary conditions:

$$H = J \sum_{i=1}^{N} \left[\sigma_i^x \sigma_{i+1}^x + \sigma_i^y \sigma_{i+1}^y + \Delta \, \sigma_i^z \sigma_{i+1}^z \right], \tag{5}$$

where J is the exchange coupling and Δ is the anisotropy parameter. Setting $\Delta = 1$ gives the isotropic Heisenberg model, while $\Delta = 0$ reduces to the XY chain. Unlike the TFIM, the model couples all three spin components, making it a more general and challenging benchmark.

2.2.2 Frustrated J_1-J_2 Heisenberg Chain

The frustrated J_1-J_2 Heisenberg chain extends the XXZ model with a next-nearest-neighbour exchange term:

$$H = J_1 \sum_i \boldsymbol{\sigma}_i \cdot \boldsymbol{\sigma}_{i+1} + J_2 \sum_i \boldsymbol{\sigma}_i \cdot \boldsymbol{\sigma}_{i+2}, \tag{6}$$

where J_1 and J_2 are the nearest- and next-nearest-neighbour coupling strengths. The competing interactions frustrate the system, preventing simultaneous minimisation of both exchange terms. Setting $J_2 = 0$ recovers the standard Heisenberg chain, while increasing J_2/J_1 drives the system into a more frustrated regime that poses a more stringent test for variational methods.

2.3 Quantum (inspired) solvers

Here we present two solver types: First, a special quantum machine designed to find the ground state of an Ising Hamiltonian, and second, a classical quantum inspired algorithm.

2.3.1 Adiabatic Quantum Computing

Adiabatic Quantum Computing is based on the adiabatic theorem, which states: *A physical system remains in its instantaneous eigenstate if a given perturbation is acting on it slowly enough and if there is a gap between the eigenvalue and the rest of the Hamiltonian's spectrum.* In this work we focus on the D-Wave Quantum Annealer, which acts according to the adiabatic theorem in the following way: In the beginning of the annealing process, the device is initiated in the ground state of a initial driver Hamiltonian H_D that can be easily constructed:

$$H_D = -\sum_{i=1}^{N} \sigma_x^{(i)} \text{ with ground state } |+\rangle^{\otimes N} \tag{7}$$

where σ_x, σ_y, and σ_z are the Pauli matrices acting on qubit i. The problem Hamiltonian is defined as

$$H_P = \sum_{i=1}^{N} = h_i \sigma_z^{(i)} + \sum_{i,j=1}^{N} J_{ij} \sigma_z^i \otimes \sigma_z^j. \tag{8}$$

During the annealing process, the system is slowly brought from H_D to H_P according to the so-called annealing schedules $A(s)$ and $B(s)$:

$$H(s) = A(s) H_D + B(s) H_P = -A(s) \sum_{i=1}^{N} \sigma_x^{(i)} + B(s) \sum_{i=1}^{N} h_i \sigma_z^{(i)} + B(s) \sum_{i=1}^{N} \sum_{j=1}^{i-1} J_{ij} \, \sigma_z^{(i)} \otimes \sigma_z^{(j)}, \tag{9}$$

where $s \in [0, 1]$ is the normalized time. $A(s)$ decreases the driving terms in H_D as s increases, and $B(s)$ activates the terms of the problem Hamiltonian H_P. At the end of the evolution, if the process happens slow enough, and assuming a noiseless environment, the system will be in the ground state of the problem Hamiltonian according to the adiabatic theorem. It is important to note that the qubits in the D-Wave Quantum Annealer are not *all-to-all* connected. The qubits are lined out in an architecture dependent grid. In the Pegasus topology, one qubit couples on average to 15 other qubits, while Zephyr qubits couple to 20 other qubits. Therefore, most problems require an additional embedding step, in which multiple physical qubits are *chained* together to form one logical qubit. For a toy-problem example of embedding a triangle shaped connectivity pattern on a QPU, see **??**. There exist three different types of couplers, connecting physical qubits:

- **Internal Couplers** connecting qubits of opposite orientation
- **External Couplers** connecting qubits aligned in parallel
- **Odd Couplers** which connect similarly aligned pairs of qubits

See **??** for visualization of different QPU topologies.

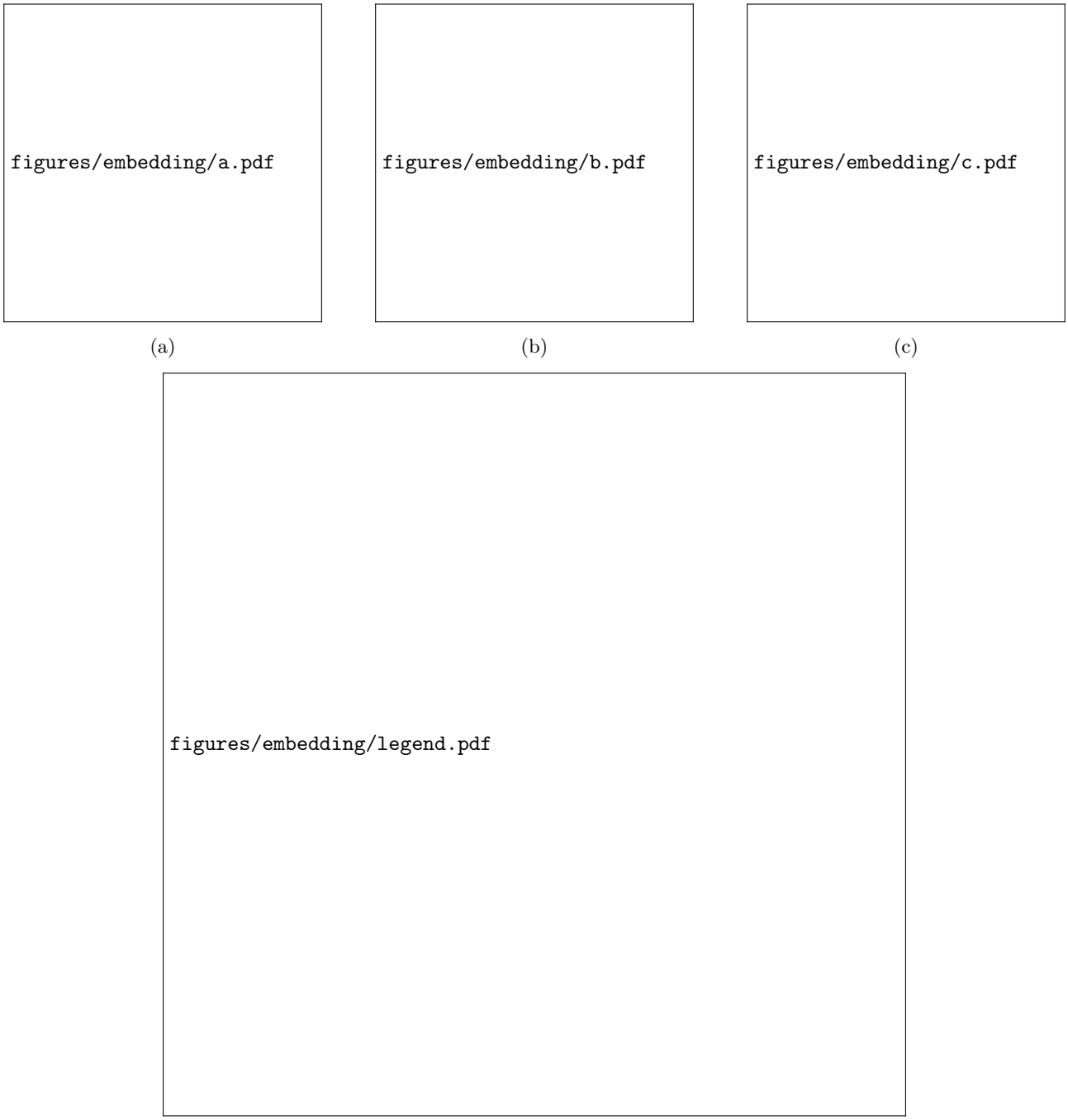
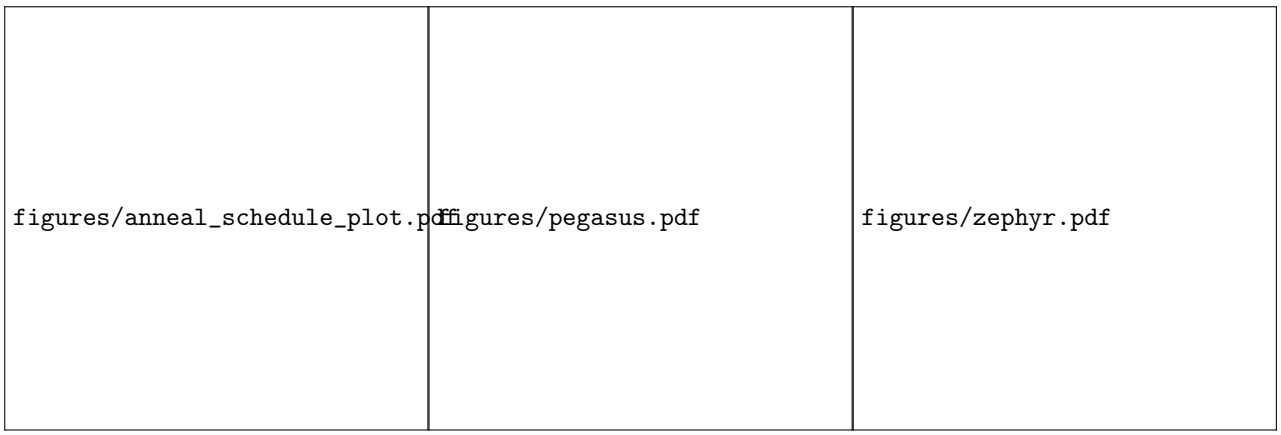


Figure 2: Minor embedding of a triangle onto a sparser hardware graph. **(a)** Logical problem graph: a triangle requires a coupler between every pair of nodes. **(b)** Target hardware graph: a 4-cycle, whose qubits admit no triangle directly. **(c)** A valid embedding: one logical node is realized as a chain of two physical qubits (joined by a strong ferromagnetic coupling, dashed), so that the three logical edges are each realized by a single physical coupler.



(a) (b) (c)

Figure 3: (a) Time-dependent energy scales executed by the D-Wave quantum annealer. The coefficient $A(s)$ (red) multiplies the transverse-field term that enables quantum tunnelling and therefore falls rapidly as the run progresses. The coefficient $B(s)$ (blue) multiplies the problem Hamiltonian and rises smoothly, so that the device moves from a tunnelling-dominated regime at $s = 0$ to a problem-dominated regime at $s = 1$. (b) Native connectivity map for the Pegasus architecture (D-Wave Advantage). Circles mark individual superconducting qubits; blue lines are internal couplers and orangle lines mark external couplers (c) Connectivity map for the newer Zephyr architecture (D-Wave Advantage2), which increases the number of couplers per qubit and introduces longer-range links.

2.3.2 Simulated Bifurcation

Simulated Bifurcation (SB) is a heuristic algorithm that numerically simulates adiabatic and ergodic (chaotic) evolutions of classical nonlinear Hamiltonian systems to solve combinatorial optimization problems. Inspired by quantum adiabatic optimization using networks of Kerr-nonlinear parametric oscillators (KPO), the algorithm encodes the ground state of an Ising Hamiltonian into the stable states of a network of classical oscillators.

Each Ising spin is represented by continuous dynamical variables, position x_i and momentum y_i, which serve as canonical conjugate variables for each oscillator. The system evolves according to coupled differential equations derived from a network of Duffing oscillators. These equations include a nonlinear x^4 term and a time-dependent pumping parameter $p(t)$. As $p(t)$ is gradually increased, the system undergoes an adiabatic bifurcation, in which a single stable equilibrium at the origin splits into two stable branches. This mechanism drives the trajectories toward configurations corresponding to low-energy states of the Ising Hamiltonian. Because all variables are updated simultaneously at each time step, the algorithm is highly parallelizable and well suited for implementation on FPGAs and GPUs.

Langevin Simulated Bifurcation Langevin Simulated Bifurcation (LSB) is a stochastic adaptation of SB designed to function as a Boltzmann sampler for training energy-based models such as Semi-Restricted Boltzmann Machines (SRBMs). While SB is a deterministic optimization method, LSB introduces stochasticity to approximate a Boltzmann distribution $B_{\beta_{\text{eff}}}$ with an unknown effective inverse temperature β_{eff}. To handle the unknown β_{eff} during learning, LSB is often combined with Conditional Expectation Matching (CEM). A detailed analysis of different temperature parameters is provided in **??**.

Conditional Expectation Matching CEM accounts for the fact that, in order to produce samples using LSB, a parameter β_{eff}, which is unknown and instance dependent, has to be found. This parameter is called the effective inverse temperature and it defines the distribution returned by the sampler. In Semi-Restricted Boltzmann Machine (SRBM), hidden variables are conditionally independent when the visible units are fixed to a condition vector r. This allows the direct calculation of the expectation value of a hidden unit:

$$\langle h \rangle_{\beta,r} = \tanh\left\{\beta\left(c_j + \sum_{i=1}^{N_v} r_i W_{ij}\right)\right\} \tag{10}$$

To estimate the expectation value, denoted $\langle h_j \rangle_{r,C}$ CEM then performs conditional LSB sampling, fixing $v = r$. In the *matching* step, CEM then minimizes the squared difference between both expectation values:

$$\beta_{\text{eff}} = \arg\min_{\beta>0} \sum_{j=1}^{N_h} \{\langle h_j \rangle_{r,C} - \langle h \rangle_{\beta,r}\}^2 \tag{11}$$

CEM can also be used with other temperature-dependent solvers, e.g. the D-Wave Quantum Annealer.

2.4 Metropolis-Hastings inspired methods

We use several Markov chain Monte Carlo (MCMC) methods to generate samples from probability distribution from which direct sampling would be difficult. All of these methods can be traced back to the Metropolis-Hastings algorithm, which we explain below.

2.4.1 Metropolis-Hastings Algorithm

The Metropolis-Hastings Algorithm is a MCMC method used to obtain samples from a (multivariate) probability distribution $P(x)$ from which sampling is difficult. The goal of the algorithm is to define a Markov chain whose stationary distribution is $P(x)$. Suppose we can evaluate an unnormalized target density $f(x)$, where $f(x) \propto P(x)$, and the normalization constant is unknown. In the **proposal** step, a *candidate* x^* from a proposal distribution $q(x^*|x_n)$ is generated, x_n being the current state of the Markov chain. In the **accept-reject** step, the acceptance probability $A(x_n \to x^*)$ is calculated[**?**]:

$$A(x_n \to x^*) = \min\left(1, \frac{f(x^*)}{f(x^n)} \times \frac{q(x_n|x*)}{q(x^*|x_n)}\right). \tag{12}$$

For the evaluation of $\frac{f(x^*)}{f(x^n)}$, knowledge of the normalization constant is not necessary. The candidate x^* is accepted with probability $A(x_n \to x^*)$.

2.4.2 Simulated Annealing

simulated annealing (SA) is the classical analogue and predecessor of quantum annealing. It works by exploring the neighborhood of a given configuration, accepting the proposed solution with a temperature dependent probability, or if the new configuration has a lower energy than the current one.

2.4.3 Gibbs sampling

Gibbs sampling can be considered a special case of the Metropolis-Hastings algorithm. It is used when sampling directly from a joint distribution is difficult, but sampling from the full conditional distributions is tractable. The algorithm constructs a Markov chain by iteratively sampling each variable (or block of variables) from its distribution conditioned on the current values of all other variables. This procedure generates a chain whose stationary distribution is the target joint distribution.

3 Variational Algorithms

Variational algorithms are a class of hybrid quantum-classical optimization methods. We focus on the Variational Quantum Eigensolver (VQE) which is designed to find the ground state of a given Hamiltonian. It consists of three parts: the *cost function* to be minimized, the *ansatz*, which in our case is a Boltzman machine, and an *expectation value estimation*. Additionally, a classical optimizer is needed to improve the ansatz at each iteration. Depending on the quantum architecture, two approaches for estimating the expectation value are possible:

- On gate-based quantum computers, measuring can be used to gather expectation values of the modified ansatz and improve it further
- In adiabatic quantum computing, the quantum device is used to generate representative samples of the modified ansatz. These samples are then used to approximate the expectation value.

As we are interested in comparing current quantum annealing devices with their classical counterparts, we focus solely on the second case. The goal is to approximate the ground state, that is, the eigenstate corresponding to the lowest eigenvalue of a Hamiltonian H. In the adiabatic case, the ansatz is defined via an energy function over visible and hidden units,

$$E_\theta(\boldsymbol{v}, \boldsymbol{h}) = \boldsymbol{a} \cdot \boldsymbol{v} + \boldsymbol{b} \cdot \boldsymbol{h} + \boldsymbol{h} \cdot \boldsymbol{W} \cdot \boldsymbol{v}, \tag{13}$$

with weights $\boldsymbol{\theta} = \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{W}\}$ respectively applied to the vector of visible units $\boldsymbol{v} = (v_1, \ldots, v_N)$ and hidden units $\boldsymbol{h} = (h_1, \ldots, h_M)$. The joint quantum state is

$$|\Psi\rangle = \sum_v \Psi(v) \, |v\rangle \,, \qquad \Psi(v) = \sqrt{\sum_h e^{-E_\theta(v,h)}}, \tag{14}$$

so that $|\Psi(v)|^2 = \sum_h e^{-E_\theta(v,h)}$ is, up to normalization, the marginal over hidden units of the joint Boltzmann distribution $p_\theta(v,h) = Z^{-1}e^{-E_\theta(v,h)}$, $Z = \sum_{v,h} e^{-E_\theta(v,h)}$, i.e. $|\Psi(v)|^2 \propto \sum_h p_\theta(v,h)$. The corresponding Born probability distribution is

$$\rho(v) = \frac{|\Psi(v)|^2}{\sum_{v'} |\Psi(v')|^2}, \tag{15}$$

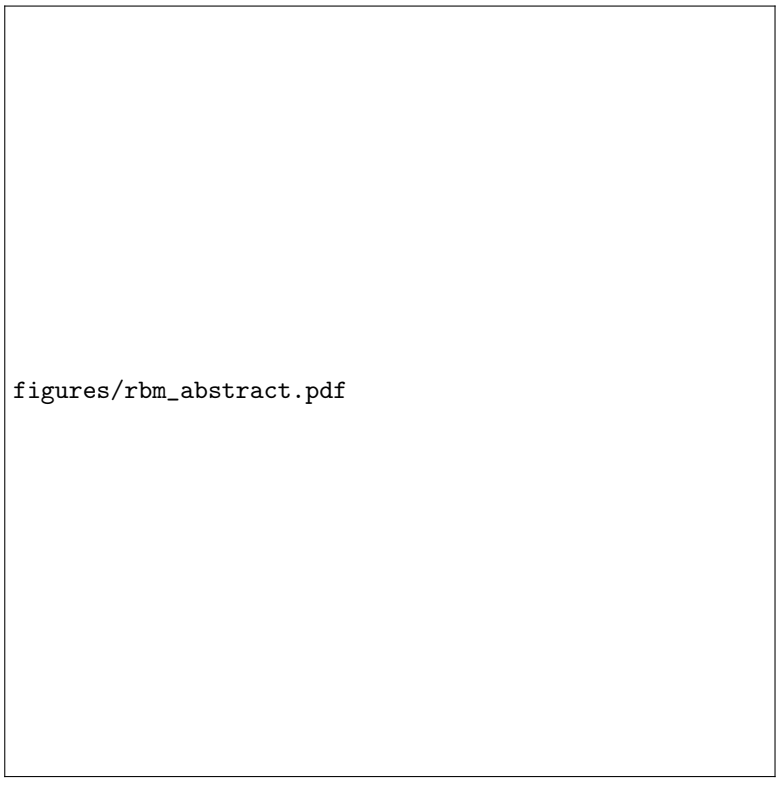
which is however intractable to normalize for large Hilbert spaces. Tracing out the hidden variables in **??**, the ansatz becomes:

$$\Psi(\mathbf{v}) = e^{-\mathbf{a}\cdot\mathbf{v}/2} \prod_{j=1}^{N_h} \left[2\cosh\left(b_j + \sum_i W_{ji} v_i \right) \right]^{1/2}. \tag{16}$$

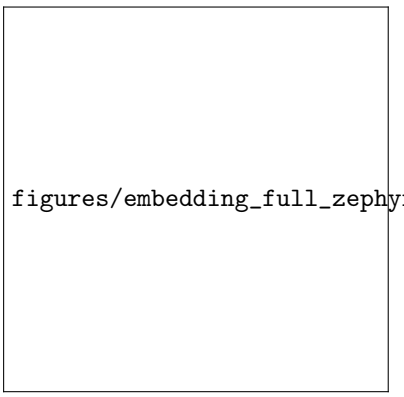
Two sampling regimes are used in this work. Classical Metropolis-Hastings sampling operates directly on the closed-form marginal $\Psi(v)$ of **??**: only the visible units are instantiated, and h never appears as an explicit random variable. Gibbs sampling on the D-Wave annealer instead targets the joint distribution $p_\theta(v,h)$: both v and h are encoded as physical qubits (**??**), the annealer returns samples of the pair (v,h), and h is subsequently discarded (marginalized) to obtain samples of v distributed as $\rho(v)$. Since D-Wave qubit biases and coupler strengths are restricted to a fixed hardware range, $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{W}$ are rescaled by a common factor β_x before being programmed onto the device; this rescaling is the origin of the effective inverse temperature β_{eff} discussed in **??**.

3.1 Boltzman machine

A restricted Boltzmann machine (RBM) is an artificial neural network consisting of visible and hidden units that form a bipartite graph. That is, no inter-layer connections are allowed. During training, the RBM learns to approximate a probability distribution. The main idea is that expectation values with respect to the quantum state can be estimated using samples drawn from the probability distribution $|\Psi(v)|^2$. Since the wavefunction ansatz is represented by a Restricted Boltzmann Machine, which corresponds to a classical stochastic Ising model, configurations v can be sampled approximately using a D-Wave quantum annealer.



(a)



(b)

Figure 4: (a) RBM with 24 visible and 24 hidden units. (b) Embedding of the corresponding bipartite graph onto the Zephyr hardware topology. Blue nodes are visible units, red nodes are hidden units, orange nodes are auxiliary chain qubits. Each physical node is connected to one auxillary qubit.

4 Experimental Analysis

We conduct experiments analyzing sampler quality and physical properties or samplers and models.

4.1 Sampling Quality

While some of the samplers are guaranteed to sample from the Boltzman distribution, others are not. In this section, we analyze the quality of samplers with regard to various parameters. One measure for sampling quality is the *total variation distance* D_{TV}. It is defined as

$$D_{\mathrm{TV}}(\mu, \nu) = \frac{1}{2} \sum_{\boldsymbol{\sigma} \in \{-1,+1\}^N} |\mu(\boldsymbol{\sigma}) - \nu(\boldsymbol{\sigma})| \,, \tag{17}$$

where μ is the histogram returned from the sampler and $\nu(\boldsymbol{\sigma}) \propto |\Psi(\boldsymbol{\sigma})|^2$ is the RBM distribution. As every sampler has a range of parameters, we are interested in how well the sampling performs with regards to the different tuning parameters. SA, Gibbs, and Metropolis can be tuned by the number of *sweeps*, that is, the number of single-spin-flip proposals, whereas LSB is tunable by the number of *steps* which updates all spins simultaneously.

We use the following workflow to determine D_{TV}: First, we use a SA pretrained RBM and compute its exact distribution $|\Psi|^2$ numerically by enumerating all possible spin configurations. We then collect samples from different samplers and compare the resulting empirical histogram to the exact distribution.

The results are presented in **??**. As a sanity check, it can be seen that Metropolis and Gibbs sampling perform theoretically near-optimal, as is expected. We observe that LSB significantly underperforms the other solvers for this small-scale example. The performance of both LSB and SA improves as the transverse field bias h increases. In **??(b)** the exact histogram of configurations $\Psi(v)|^2$ is plotted. For $h = 0.5$ the distribution is peaked around a handful of highly probable sample configuration, whereas the distribution becomes more uniform for stronger values of h. This explains the performance increase for LSB and SA samplers.

The peaked distribution originates from the fact that the spins in the TFIM description given by **??** are aligned for low h values (*ferromagnetic* phase). For $h \to \infty$ the transverse field dominates and the ground state becomes a product of single-spin σ_x eigenstates. Measured in the z-basis, this leads to a uniform distribution in the thermodynamic limit.

4.2 Sampling temperature & CEM

In this section, we evaluate the influence of temperature on different models and solvers. We focus on two quantities: the rescaling factor β_x and the effective temperature β_{eff}. Sampling from an ansatz, all weights in the RBM are rescaled by β_x, that is, the sampler is programmed with $E_{\mathrm{in}} = \alpha E_\theta$, $\alpha = 1/\beta_x$. If the device samples at hardware inverse temperature β_{hw}, it draws from $p(v,h) \propto e^{-\beta_{\mathrm{hw}} E_{\mathrm{in}}(v,h)} = e^{-(\alpha\beta_{\mathrm{hw}}) E_\theta(v,h)}$, i.e. relative to the unscaled model $\beta_{\mathrm{eff}} = \alpha\beta_{\mathrm{hw}}$. For an ideal Gibbs sampler held at $\beta_{\mathrm{hw}} = 1$, the following equation holds:

$$\beta_{\mathrm{eff}} = \frac{1}{\beta_x}. \tag{18}$$

4.2.1 Langevin Simulated Bifurcation

As LSB is a temperature dependent solver and not guaranteed to sample from the Boltzmann distribution, it can be used to illustrate the influence of the input temperature β_x on the solution quality. Contrary to the quantum annealer, sampling using LSB does not involve additional costs. To evaluate the inpact of the scaling parameter β_x, we calculate the total variational distance D_{TV} between the samples and the precise Boltzmann distribution by numerical enumeration. **??** clearly illustrates that the sampler temperature is problem dependent. D_{TV} is minimal at the point where the effective temperature β_{eff} is 1.

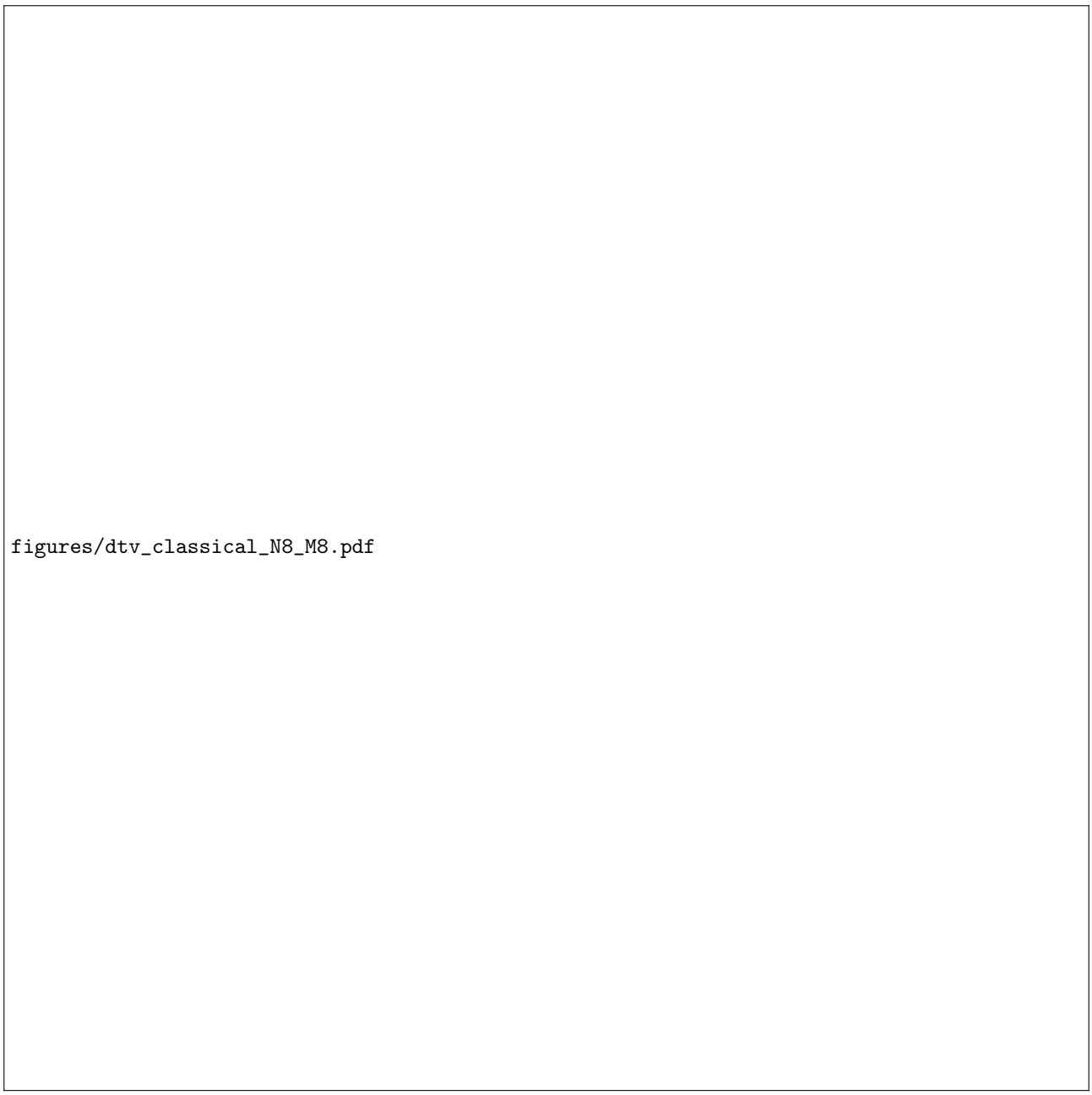
4.2.2 CEM

We have seen that correctly choosing the sampling temeperature is crucial in obtaining meaningful distributions. CEM[**?**] is a technique to dynamically estimate the temperature based on conditionally drawn samples. The hidden units' conditional expectation value

$$\langle h_j \rangle = \tanh\left(\beta \cdot \Theta_j\right), \qquad \Theta_j = b_j + \sum_i v_i W_{ij} \tag{19}$$

for some fixed visibile units vector. β_{eff} is then given by finding (*matching*) the optimal β,

$$\beta_{\mathrm{eff}} = \arg\min_\beta F(\beta), \text{ with } F(\beta) = \sum_j ((h_{j,\mathrm{observed}} - \tanh(\beta \cdot \Theta_j))^2 \tag{20}$$



(a) Total variation distance as a function of sampling effort.

(a) Total variation distance D_{TV} between the LSB empirical histogram and the exact distribution $|\Psi(v)|^2$. The horizontal dashed line marks the sampling floor.

(b) Effective inverse temperature β_{eff} of the LSB samples, estimated by minimizing the total variational distance between the samples and the actual distribution. The dashed curve shows the ideal $1/\beta_x$ prediction for a perfect Gibbs sampler, the horizontal dotted line marks the VMC target $\beta_{\mathrm{eff}} = 1$.

Figure 6: Influence of the LSB energy-scale parameter β_x on sampling quality for a trained TFIM RBM with $N = 8$ visible spins. The vertical dotted line in each panel marks the β_x that minimises D_{TV}.

, where $h_{j,\mathrm{observed}}$ are actual samples of the hidden unit j drawn from the sampler.

The matching step is visualized in **??**, where the left subfigure shows multiple candidate functions $\tanh(\beta\Theta)$ dependent on β that can be used to match the returned samples. The right subfigure describes the corresponding minimization, in which the squared loss function $F(\beta)$ is minized over β (in practice using scipy's `minimize_scalar` function). This single fit at $N = 16$ is illustrative of the matching procedure; a systematic validation of CEM against exact maximum-likelihood or total-variation estimates across sizes, fields, parameter sets, and training iterations, including bias, variance, and confidence intervals, remains to be established.

figures/cem_matching_candidates.pdf

(a) Candidate fits vs. observed hidden units.

figures/cem_matching_objective.pdf

(b) Matching objective $F(\beta)$.

Figure 7: The CEM matching step for $N = 16$ spins with $h = 1.0$. (a) Several candidate $\tanh(\beta\Theta)$ curves against the empirically observed hidden-unit samples. $\beta_{\mathrm{eff}} = 1.94$ hugs the data and represents the best estimate of the sampler's temperature. (b) The sqaured loss function to be minimized

4.3 Limitations of variational methods

In this section we describe computational and theoretical limitations to using variational methods in a hybrid quantum-classical setup.

4.3.1 The Sign Problem

It is well known that capturing quantum dynamics with non-trivial sign structure using a real-valued variational ansatz is computationally hard (*sign problem*). This does not pose a problem for the TFIM models, as its ground state has only positive amplitudes. More complex models, such as some J1J2-Heisenberg models on the other hand cannot be simulated using our methods. The reason for this is that the factors $e^{-\mathbf{a}\cdot\mathbf{v}/2}$ and $\cosh(\cdot)$ of the ansatz defined in **??** are strictly positive for real parameters and therefore the ansatz does not allow for the representation of negative amplitudes.

The Marshall sign rule For certain Heisenberg models with bipartite lattice structure, the Marshall sign rule can be used to remove the non-trivial sign structure of the ground state wavefunction by basis transformation.

Considering the following unitary transformation on bipartite lattices $A \cup B$:

$$U = \prod_{i \in A} e^{i\pi S_i^z},$$

the ground state wavefunction can be written as

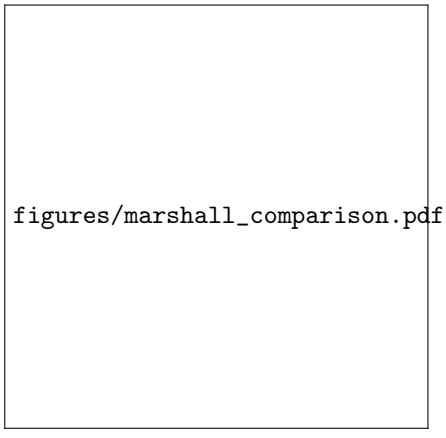
$$U|\psi_0\rangle = \sum_{\sigma} |c_\sigma|\,|\sigma\rangle,$$

with only positive amplitudes. We measure the impact of the sign problem on a given probability distribution by the average sign[**?**]. It is given by:

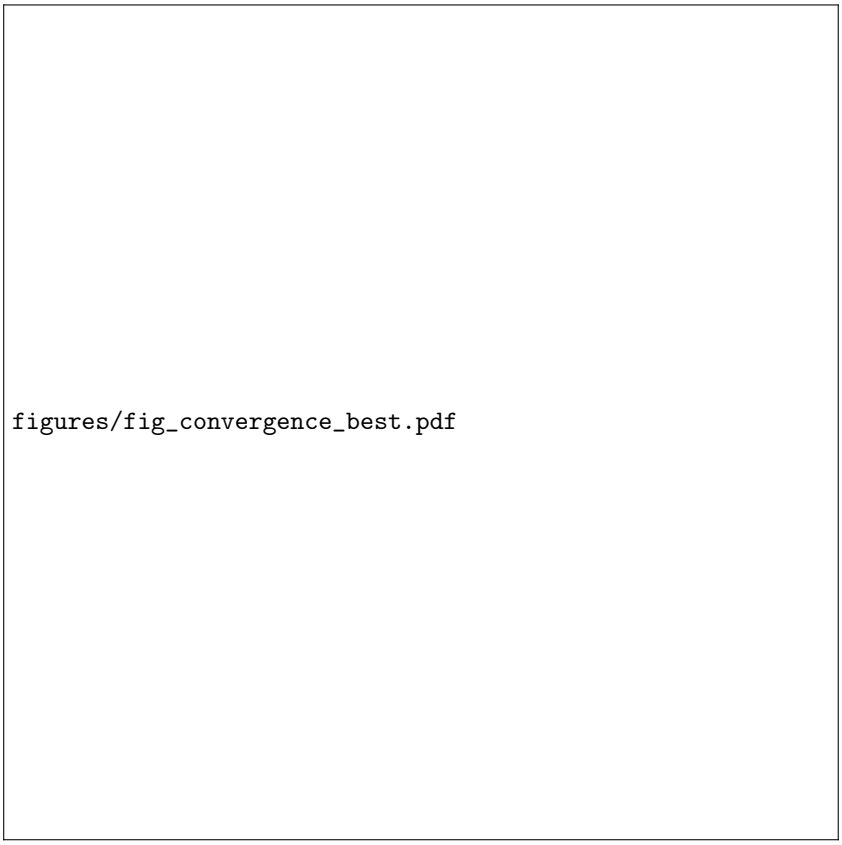
$$\langle s \rangle = \frac{\sum_{\mathbf{v}} \Psi(\mathbf{v})}{\sum_{\mathbf{v}} |\Psi(\mathbf{v})|} \tag{21}$$

Interestingly, we empirically observe that even for frustrated Heisenberg models with $0 \leq \frac{J_2}{J_1} \leq 0.5$ the RBM with the Marshall sign correction can accurately approach the ground state energy, whereas without the correction the variational energy degrades significantly (see **??**).

Figure 8: Spin configurations sampled from a trained RBM-VMC model for the one-dimensional Transverse-Field Ising Model (TFIM) at three values of the transverse field h. **Top row:** each column shows 2 000 spin configurations $\boldsymbol{\sigma} \in \{-1, +1\}^N$ sampled from the learned wave function $|\Psi_\theta|^2$, sorted by total magnetisation. **Bottom row:** spin-spin correlation $C(r) = \langle \sigma_0^z \sigma_r^z \rangle$ as a function of distance r, computed from VMC samples (solid line) and exact diagonalisation (dashed line). The relative energy error $\varepsilon = |E_{\mathrm{VMC}} - E_{\mathrm{exact}}|/|E_{\mathrm{exact}}|$ is shown in each panel. The model is a fully-connected RBM ($N = N_h = 16$) trained for 300 iterations of Stochastic Reconfiguration with 500 Metropolis-Hastings samples per step.



(a) Sign impurity ϵ (top) and theoretical energy floor for any real positive ansatz vs. best RBM error (bottom), computed by exact enumeration at four values of J_2/J_1 for $N = 8$ and $N = 12$.



(b) Best RBM convergence curves (E/N vs. SR iteration) for $N \in \{8, 12\}$ and $J_2/J_1 \in \{0.3, 0.5, 0.7, 0.9\}$. The red dashed line marks the exact ground-state energy E_{exact}/N.

Figure 9: Visualization of the sign problem on the J_1–J_2 Heisenberg chain. Left: sign impurity and overall training dynamics, Right: best run for selected instances.

4.4 Improving D-Wave Topology performance

A crucial factor in the success of an annealing progress is the *problem embedding* on the chip. Here, we assess two aspects related to embedding: First, introducing sparsity to fit the RBM natively on the chip, and, second, embedding the problem multiple times to save computing time.

4.4.1 Sparse embedding

As described in **??**, problems implementable on quantum annealer often need to be fit to the QPU in an additional embedding step. We are interested in whether a sparse, but native RBM, that is, one that fits natively on the QPU, performers reasonably. The sparse RBM is creating according to the following algorithm: First, constructe a bipartite graph by keeping only edges that correspond to internal couplers on the D-Wave QPU. One set of the graph corresponds to visible RBM units, the other one to hidden units. Then, greedily pick qubits for the visible units, by maximizing the overlap between neighbor sets of visible units. After all visible units are chosen, create a minimum set-cover by picking the fewest hidden qubits such that every visible qubit has at least one hidden neighbor. The remaining hidden units are filled by best connected qubits. See also **??** for a visual overview of the algorithm. We also looked directly at the effect of sparsity on its own, keeping the number of hidden units equal to the number of visible units and only changing how many of the real couplers are removed from the connectivity mask. When removing couplers, we always removed one from the hidden unit with the highest remaining degree, so that no visible or hidden unit was ever left with zero connections. **??** shows the resulting energy error as a function of sparsity, for both classical training and training with samples from the real QPU. In both cases, error increases sharply as sparsity increases, and the real QPU curve gets worse faster than the classical one at high sparsity — real hardware sampling adds an extra penalty on top of the sparsity itself.

4.4.2 Parallel Embeddings

To speed up calculation and save computing time, it is possible to embed multiple smaller, independent problems on the QPU. For example, it is possible to gather 1000 independent samples using only 250 annealing runs, by embedding the RBM four times on the QPU. We experimentacally validate that this approach produces meaningful results, as can be seen in **??**. The left subfigure visualizes a possible parallel embedding. In the right subfigure, it can be seen that parallel approaches convergence faster to the ground state, using less QPU time.

5 Implementation

In this chapter, we are going to describe the conceptual and technological aspects of implementing variational algorithms in a hybrid quantum / classical setup.

5.1 Stochastic gradient descent versus Stochastic reconfiguration

Traditionally, neural networks are trained using gradient descent. We will not go into details about this well known approach, as it is extensively covered by literature. Gradients are computed efficiently using backpropagation which exploits the chain rule, reusing intermediate derivatives to evaluate the compositions of continuous functions. For most of this article, we use the stochastic reconfiguration (SR) method introduced in [**?**]. The main advantage of SR over gradient descent is that it exploits more knowledge about the ansatz than gradient descent. In particular, gradient descent lives in the *parameter space*, and does not take into account the influence of parameter changes on the actual wave function *probability space* $|\Psi|^2$.

5.1.1 Stochastic reconfiguration

SR is implemented as follows: given the logarithmic derivative for each parameter k, that is the gradient observable

$$O_k(\mathbf{v}) = \frac{\partial \log \Psi(\mathbf{v})}{\partial \theta_k}, \tag{22}$$

with $\boldsymbol{\theta} = \{\mathbf{a}, \mathbf{b}, W\}$, the energy gradient or force vector is defined as

$$F_k = \langle O_k E_{\text{loc}} \rangle - \langle O_k \rangle \langle E_{\text{loc}} \rangle, \tag{23}$$

which is the covariance of the gradient observable and the local energy. The local energy $E_{\text{loc}}(\mathbf{v})$ estimates the variational energy $\langle H \rangle$ of the current RBM ansatz using samples drawn from $|\Psi(\mathbf{v})|^2$:

$$E_{\text{loc}}(\mathbf{v}) = \frac{\langle \mathbf{v}|H|\Psi\rangle}{\langle \mathbf{v}|\Psi\rangle} = \sum_{\mathbf{v}'} H_{\mathbf{v}\mathbf{v}'} \frac{\Psi(\mathbf{v}')}{\Psi(\mathbf{v})} \tag{24}$$

Figure 10: Construction of a chain-free sparse RBM from a D-Wave Zephyr hardware graph ($N = 8$ visible units). **(a)** Identify internal couplers (green) and external/odd couplers (purple)**(b)** Creation of bipartite graph by restricting to internal couplers, splitting into hidden unit candidates (red) and visible units (blue). **(c)** RBM's visible-hidden connectivity mask by selecting the hidden units which maximize connectivity while keeping a min-set-cover. Non-selected candidates (grey) are dropped.

(a) (b)

Figure 11: Error as a function of sparsity, with the number of hidden units kept equal t in both plots. **(a)** Error for different values of h. **(b)** Error at $h = 1$, comparing classical training to training on the real QPU. The dotted line marks the lowestllows at this size.

The next step then is to define the quantum geometric tensor S, the sample covariance of the O_k vectors, which measures how sensitive the Born distribution $|\Psi|^2$ is to each parameter direction. In matrix form, if $\bar{O}$ is the matrix of mean-centred gradients, then $S = \frac{1}{N_s}\bar{O}^\top \bar{O}$. After solving the SR system

$$S \cdot \delta\theta = F \tag{25}$$

for $\delta\theta$, the parameters are updated by

$$\theta \leftarrow \theta - \eta\, \delta\theta, \tag{26}$$

with learning rate η.

Algorithm 1 Stochastic Reconfiguration

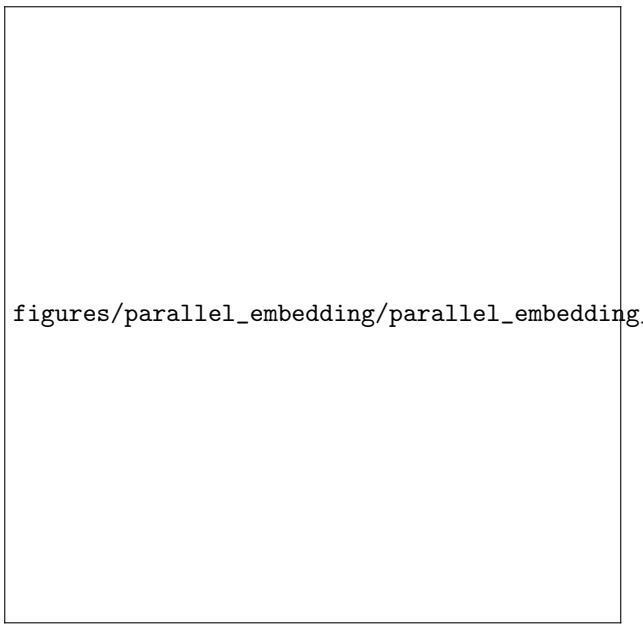
Require: Parameters $\boldsymbol{\theta} = \{\mathbf{a}, \mathbf{b}, W\}$, learning rate η, regularisation λ

for $t = 1, 2, \ldots$ **do**
  Sample $\mathbf{v}^{(s)} \sim |\Psi_{\boldsymbol{\theta}}|^2$ for $s = 1, \ldots, N_s$ ▷ MCMC
  $E_{\mathrm{loc}}(\mathbf{v}^{(s)}) \leftarrow \sum_{\mathbf{v}'} H_{\mathbf{v}\mathbf{v}'}\, \Psi(\mathbf{v}')/\Psi(\mathbf{v})$ for each s
  $O_k(\mathbf{v}^{(s)}) \leftarrow \partial \log \Psi(\mathbf{v}^{(s)})/\partial\theta_k$ for each s, k
  $F_k \leftarrow \langle O_k E_{\mathrm{loc}}\rangle - \langle O_k\rangle\langle E_{\mathrm{loc}}\rangle$
  $S \leftarrow \frac{1}{N_s}\bar{O}^\top \bar{O} + \lambda I$
  Solve $S\, \delta\boldsymbol{\theta} = \mathbf{F}$ ▷ conjugate gradient, matrix-free
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\, \delta\boldsymbol{\theta}$
end for

5.1.2 Conjugate gradient

As $S \in \mathbb{R}^{N_{\mathrm{params}} \times N_{\mathrm{params}}}$ grows quadratically with the number of parameters $N_{\mathrm{params}} = N + M + NM$, with N and M the number of visible and hidden units respectively, solving the SR system in **??** is intractable for large system sizes. We therefore apply a technique called conjugate gradient (CG), which requires only matrix vector products $S \cdot x$ and converges for any symmetric positive-definite matrix.Computing the matrix–vector product then costs only $\mathcal{O}(N_s \cdot N_{\mathrm{params}})$:

$$S \cdot x = \frac{1}{N_s}\bar{O}^\top(\bar{O}\, x) + \lambda x \tag{27}$$



(a) Four independent $K_{8,8}$ RBM embeddings ($n_{\text{parallel}} = 4$) placed simultaneously on the Pegasus chip. Colors distinguish copy 1-4; hatched markers denote visible units, solid markers denote hidden units.

(b) Energy vs. cumulative QPU access time for $n_{\text{parallel}} \in \{1, 3, 5\}$ (best of 3 seeds each). Higher n_{parallel} reaches the converged energy plateau using less real QPU time.

Figure 12: Parallel embedding on a D-Wave QPU: (a) physical embedding layout, (b) resulting training speedup.

5.2 Exact Solving

Determining the exact value of the ground state up to numerical precision is crucial for evaluating the performance of the trained models. For TFIM, exact solving is possible for arbitrary system sizes, whereas solving the J1J2 Heisenberg model is computationally hard.

5.2.1 TFIM ground state solving in $O(N)$

By transforming spin operators to fermionic operators using the Jordan-Wigner transformation and diagonalizing them in the fermionic basis, the 1d TFIM can be solved exactly in $O(N)$. The difference between spins and fermions is that spin operators on different sites need to commute, whereas the fermion's creation and annhiliation operators must anticommute. Spin operators can be mapped to fermionic operators in the following way:

$$c_j = \left(\prod_{k<j} \sigma_k^z \right) \sigma_j^- \tag{28}$$

$$c_j^\dagger = \left(\prod_{k<j} \sigma_k^z \right) \sigma_j^+ \tag{29}$$

After transformation, the TFIM from **??** can be expressed as quadratic fermion Hamiltonian

$$H = -J \sum_j (c_j^\dagger - c_j (c_{j+1} \dagger + c_{j+1}) - h \sum_j (1 - 2 c_j^\dagger c_j). \tag{30}$$

Bogugliev transformation allows us to calculate the energy needed to excite a single quasiparticle with momentum k[**?**]:

$$\epsilon(k) = \sqrt{J^2 + h^2 - 2Jh\cos(k)}. \tag{31}$$

The ground state energy is then the sum of all quasiparticle energies taken negative[**?**],

$$E_0 = -\sum_k \epsilon(k), \tag{32}$$

since in the Bogoliubov vacuum every mode contributes $-\epsilon(k)$ to the vacuum energy.

With periodic boundary conditions on the spin chain, the allowed momenta depend on the total fermion parity. There are two possible parity sectors, leading to k from a periodic or anti-periodic boundary condition[?]

$$k_m^{(\text{even})} = \frac{\pi(2m+1)}{N}, \quad m = 0, \ldots, N-1 \quad \text{(even parity)}, \tag{33}$$

$$k_m^{(\text{odd})} = \frac{2\pi m}{N}, \quad m = 0, \ldots, N-1 \quad \text{(odd parity)}. \tag{34}$$

where N_f is the total number of fermions. Since the parity of the ground state depends on the system parameters, both sectors are evaluated and the true ground state energy is taken as the minimum. The ground state energy is the lower of the two sectors,

$$E_0 = \min\left(-\sum_m \epsilon\left(k_m^{(\text{even})}\right), \; -\sum_m \epsilon\left(k_m^{(\text{odd})}\right) \right), \tag{35}$$

which sector gives the lower ground state energy depends on N and h. This gives an exact result for any system size N in $O(N)$ time.

5.2.2 Exact Diagonalization

In cases where mapping to a quadratic fermion Hamiltonian is not possible, e.g. in the case of long range interactions, exact diagonalization is used for small system sizes N. Exact diagonalization works by enumerating all 2^N basis states of the Hilbert space, constructing the Hamiltonian as a $2^N \times 2^N$ matrix in that basis, and finding its lowest eigenvalue via the Lanczos algorithm. Lanczos algorithm avoids diagonalizing the entire matrix and only returns the matrix' lowest eigenvalue. We use `scipy`'s `eigsh` implementation of this algorithm.

5.2.3 Other methods

The lowest energy returned by variational models using Gibb's sampling can be regarded as an upper bound to the ground state, as it can never be lower than the ground state due to the variational principle.

5.3 JAX

(Quantum) variational algorithms do not only strain *quantum* ressources, but also push *classical* computers to their limits. Notably, at each iteration, the stochastic reconfiguration process described in **??** adapts all parameters of the weights and biases. We use several techniques in this project to speed up this process. JAX, implementing the *functional programming* paradigm, allows for the speed up of function calls and linear algebra caluclation by offering features such as just-in-time (JIT) compilation and XLA (accelerated linear algebra). We will describe the features of JAX in detail and give examples from our codebase.

5.3.1 Just-in-time compilation

JAX transformation require function to be *functionally pure*. To summarize, a function is considered functionally pure, if its output depends only on the function's input parameters, and all output is passed through the return variable. As an example, a function that involves `print()` statements or writes to global variables is not considered to be functionally pure, as its output does not exclusively pass through the `return` statement. JAX does not guarantee the proper behavior of impure functions. During JIT compilation, the function is transformed to primitives, which in JAX are defined to be "fundamental units of computation"[?]. As an example, the following function header is used to calculate the local energy (**??**):

```
@functools.partial(jax.jit, static_argnums=(4, 5))
def _local_energy_1d_jit(
    V: jax.Array,   # (ns, N) -- spin configs
    W: jax.Array,   # (N, M)  -- RBM weights
    a: jax.Array,   # (N,)    -- visible biases
    b: jax.Array,   # (M,)    -- hidden biases
    h: float,       # static
    N: int,         # static
) -> jax.Array:     # (ns,)   local energies
    ...
```

h and N are passed as static parameters, meaning that they will be hard-coded in the compiled function after JIT. Jax compiles a kernel for each combination of static variables; as h and N never change during a training run, this means that the function is only compiled once.

5.3.2 Vectorized Map

`jax.vmap` takes a function written for a single input and returns a new function that maps it over a leading batch axis as a single vectorised kernel.

As a concrete example, consider `log_psi_fn` defined for a single spin configuration $\mathbf{v} \in \{-1, +1\}^N$. Evaluating it over all n_s sampled configurations reduces to:

```
log_p_V = jax.vmap(log_psi_fn)(V)     # V: (ns, N) -> log_p_V: (ns,)
```

XLA compiles this as a single operation over the batch dimension rather than n_s independent kernel launches. The resulting operation can be compiled using JIT for faster execution:

```
batched_f = jax.jit(jax.vmap(log_psi_fn))
```

5.3.3 Autograd

Jax offers automatic differentation to compute function gradients. However, we do not make use of this feature as gradients are calculated analytically (see [**?**]).

5.4 Hardware and Software Configuration

D-Wave experiments use the `Advantage_system6` (Pegasus) and `Advantage2_system1` (Zephyr) solvers. Forward-anneal runs use a $20\,\mu s$ anneal time and `num_reads` equal to the requested sample count, with `auto_scale` enabled and D-Wave's default chain strength unless stated otherwise; embeddings are cached per `(n_visible, solver)` using `minorminer`'s biclique embedder. Classical simulated-annealing baselines use `dwave-neal` with 1000 sweeps and a geometric β schedule over $[0.01, 10]$. FPGA/VeloxQ runs use `Float32` precision, a 100 MHz core clock, 1024 parallel replicas, and 1000 update steps per run. All stochastic components (JAX/NumPy) are seeded per run via `np.random.randint` feeding `jax.random.PRNGKey`.

6 Appendix

Claim. Given the regularised quantum geometric tensor $S = \frac{1}{N_s}\bar{O}^\top\bar{O} + \lambda I$, its product with any vector $x \in \mathbb{R}^{N_{\text{params}}}$ satisfies

$$S\,x = \frac{1}{N_s}\bar{O}^\top(\bar{O}\,x) + \lambda x. \tag{36}$$

Derivation.

Step 1: Write S in sample form. By definition, $S_{kl} = \langle O_k O_l\rangle - \langle O_k\rangle\langle O_l\rangle$. Introducing the matrix $O \in \mathbb{R}^{N_s\times N_{\text{params}}}$ with entries $O_{sk} = O_k(\mathbf{v}^{(s)})$, the sample estimates are

$$\langle O_k O_l\rangle = \frac{1}{N_s}\sum_s O_{sk}\,O_{sl} = \frac{1}{N_s}(O^\top O)_{kl}, \qquad \langle O_k\rangle\langle O_l\rangle = \mu_k\mu_l, \tag{37}$$

where $\mu_k = \frac{1}{N_s}\sum_s O_{sk}$ is the sample mean of the k-th observable.

Step 2: Express S via the mean-centred matrix. Define $\bar{O} \in \mathbb{R}^{N_s\times N_{\text{params}}}$ by $\bar{O}_{sk} = O_{sk} - \mu_k$. Then

$$\frac{1}{N_s}(\bar{O}^\top\bar{O})_{kl} = \frac{1}{N_s}\sum_s (O_{sk}-\mu_k)(O_{sl}-\mu_l) = \langle O_k O_l\rangle - \mu_k\mu_l = S_{kl} - \lambda\delta_{kl}, \tag{38}$$

so, including regularisation,

$$S = \frac{1}{N_s}\bar{O}^\top\bar{O} + \lambda I. \tag{39}$$

Step 3: Apply to x. Since matrix multiplication is associative,

$$S\,x = \frac{1}{N_s}\bar{O}^\top\bar{O}\,x + \lambda I\,x = \frac{1}{N_s}\bar{O}^\top\underbrace{(\bar{O}\,x)}_{z\,\in\,\mathbb{R}^{N_s}} + \lambda x. \qquad \square \tag{40}$$

Step 4: Two-pass interpretation. The product Sx therefore decomposes into two operations:

1. **Forward pass.** Compute $z = \bar{O}\,x \in \mathbb{R}^{N_s}$, where $z_s = \sum_k \bar{O}_{sk}\,x_k$ is the projection of x onto the centred gradient of sample s. Cost: $\mathcal{O}(N_s \cdot N_{\text{params}})$.
2. **Backward pass.** Compute $\bar{O}^\top z \in \mathbb{R}^{N_{\text{params}}}$, where $(\bar{O}^\top z)_k = \sum_s \bar{O}_{sk}\,z_s$ back-projects the per-sample scalars into parameter space. Cost: $\mathcal{O}(N_s \cdot N_{\text{params}})$.

The matrix $S \in \mathbb{R}^{N_{\text{params}}\times N_{\text{params}}}$ is never formed. Total cost per matrix–vector product: $\mathcal{O}(N_s \cdot N_{\text{params}})$, identical to evaluating the gradient once.